

AI + ML ENGINEER MASTERY ROADMAP

A sequential, no-duplication learning plan focused on top-company interviews and production-grade AI systems.

Personalized for Sai Ruthwik Reddy Kandimalla

September 2026 | Target: AI + ML Engineer / Applied AI
Engineer / GenAI Engineer

Core principle: Do not repeat basic LLM, RAG, LangChain, CrewAI, QLoRA, multimodal AI, or introductory agent concepts. Finish only the missing advanced agentic modules, then build depth in engineering, ML, deployment, and system design.

How to Use This Roadmap

This plan starts from your current position. It assumes that you completed the Ed Donner AI Engineer LLM track and reached CrewAI in the Agentic AI track. It deliberately avoids re-teaching concepts you already covered.

Recommended operating rhythm: 60% building, 25% structured learning, 15% interview revision. Every phase ends with a concrete artifact, not only a certificate.

Already covered - do not restart

- Frontier and open-source model usage
- Hugging Face basics, LangChain basics, Gradio and LLM APIs
- Basic RAG pipelines and embeddings
- QLoRA fine-tuning workflow
- Prompting, tool calling and introductory agents
- OpenAI Agents SDK and CrewAI fundamentals
- Multimodal and LLM product demonstrations
- Basic deployment shown inside the completed LLM course

Still complete from your current Agentic AI track

- LangGraph module and project
- AutoGen module for comparative framework awareness
- MCP module and final capstone
- Rebuild at least one project without referring to the instructor code

Master one primary stack

Agent orchestration	LangGraph	Stateful, controlled, recoverable workflows
API	FastAPI + Pydantic	Production service interface
Data	PostgreSQL + pgvector + Redis	Application data, retrieval, cache and state
Cloud	AWS	Deployment, secrets, logs and managed infrastructure
ML/DL	scikit-learn + PyTorch	Prediction systems and model understanding

Roadmap at a Glance

0	Finish missing agentic modules	3-4 weeks	LangGraph + MCP capstone
1	Advanced Python and engineering	5-6 weeks	Tested modular Python service
2	DSA and interview coding	Ongoing 4-5 months	120 curated problems
3	SQL, PostgreSQL and data layer	3-4 weeks	Database-backed API
4	FastAPI and backend engineering	4 weeks	Authenticated streaming API
5	Classical ML and mathematics	8-10 weeks	End-to-end ML project
6	PyTorch and DL foundations	7-9 weeks	Train and serve a DL model
7	Production infrastructure and AWS	6-8 weeks	Docker + CI/CD + cloud deployment
8	Production agent security and reliability	5-7 weeks	Secure, evaluated agent service
9	System design and interview sprint	4-6 weeks	Interview portfolio and design pack

Realistic horizon: about 9-12 months at 4-6 focused hours per day. Start applying before the roadmap ends, once you have one deployed flagship project and can confidently handle Python, RAG/agents, APIs, Docker, SQL, and system-design fundamentals.

Phase 0 - Finish Only the Missing Agentic AI Content

Suggested duration	3-4 weeks
Outcome	Complete your unfinished course without restarting topics already learned.

Learn in this order

- Finish LangGraph: state schemas, nodes, conditional edges, checkpointing, interrupts, human approval and streaming
- Finish AutoGen to understand its collaboration model; do not aim for equal mastery
- Finish MCP: client-server architecture, tools, resources, prompts, transport, authentication and permission boundaries
- Complete the capstone and rebuild one workflow with your own domain and data

Evidence of mastery

- A LangGraph workflow with persistent state, at least three tools, structured output and human approval
- An MCP server exposing two safe tools, connected to your agent
- A README explaining why LangGraph was chosen over a normal deterministic workflow

Selective use of your planned courses

- Krish Naik LangGraph course: skip Python, basic LangChain, basic prompts, basic RAG and basic tool calling. Study advanced state, persistence, subgraphs, error recovery, testing, streaming and deployment.
- AI Security Bootcamp: skip introductory LangChain/LangGraph repetition. Study guardrails, evaluation, observability, gateways, Redis, red teaming, reliability and architecture.

Phase 1 - Advanced Python and Software Engineering

Suggested duration	5-6 weeks
Outcome	Write maintainable production code rather than notebook-only demos.

Learn in this order

- Type hints, dataclasses and Pydantic models
- Decorators, iterators, generators and context managers
- async/await, concurrency, timeouts and cancellation
- Logging, configuration, secrets and dependency management with uv
- SOLID principles, dependency injection and common design patterns
- pytest, fixtures, mocking, unit tests, integration tests and end-to-end tests
- Profiling, debugging, linting and formatting

Evidence of mastery

- Convert one existing agent notebook into a package with agents/, tools/, services/, api/, config/ and tests/
- Achieve meaningful tests for tools, routing, failures and structured outputs
- Mock external model calls so tests run quickly and cheaply

Exit test: given a blank repository, you can create a virtual environment, package structure, configuration system, logging, tests, and a working async service without copying a tutorial.

Phase 2 - DSA and Interview Coding

Suggested duration	45-60 minutes daily for 4-5 months
Outcome	Pass coding screens and explain efficient Python solutions.

Learn in this order

- Complexity analysis
- Arrays, strings, hash maps and sets
- Stacks, queues, linked lists and heaps
- Sorting, binary search, two pointers and sliding window
- Recursion, backtracking, trees, BFS and DFS
- Graphs and basic dynamic programming
- Practical parsing, JSON transformation, API response handling and log processing

Evidence of mastery

- Solve about 50 easy and 70 medium curated problems
- Explain brute-force and optimized approaches aloud
- Write tests for edge cases and state time/space complexity

Do not pause the technical roadmap until DSA is complete. Run DSA in parallel as a daily interview habit.

Phase 3 - SQL, PostgreSQL and the Data Layer

Suggested duration	3-4 weeks
Outcome	Design and query the storage layer of a production AI application.

Learn in this order

- SQL joins, aggregation, subqueries, CTEs and window functions
- Indexes, transactions, normalization, constraints and query plans
- PostgreSQL schema design and migrations
- pgvector: embeddings, metadata filtering and similarity search
- Redis: caching, rate limiting, session/checkpoint storage
- S3-style object storage and document metadata
- Tenant isolation and access-controlled retrieval

Evidence of mastery

- Database schema for users, conversations, documents, traces, feedback and evaluations
- Queries that produce usage, cost and quality reports
- A FastAPI-compatible repository layer with transaction-safe operations

Phase 4 - FastAPI and Backend Engineering

Suggested duration	4 weeks
Outcome	Expose AI workflows as reliable, secure, asynchronous services.

Learn in this order

- HTTP, REST, status codes and API contracts
- FastAPI routes, Pydantic validation and dependency injection
- Async endpoints, streaming, Server-Sent Events and background tasks
- JWT/OAuth fundamentals, authorization and role-based access
- Middleware, request IDs, rate limiting and exception handling
- OpenAPI documentation, API versioning, health/readiness endpoints
- Webhooks and queue-based long-running jobs

Evidence of mastery

- POST /chat, POST /documents, POST /feedback, GET /conversations/{id}, GET /health and GET /metrics
- Streaming agent responses with cancellation and timeout handling
- Authentication, authorization, validation and consistent error responses

Phase 5 - Classical ML and Applied Mathematics

Suggested duration	8-10 weeks
Outcome	Become an AI + ML Engineer, not only an LLM framework developer.

Learn in this order

- NumPy, Pandas, visualization and exploratory data analysis
- Statistics: distributions, sampling, confidence intervals, hypothesis tests and Bayes intuition
- Linear algebra: vectors, matrices, dot products and PCA intuition
- Calculus: derivatives, gradients and gradient descent intuition
- Preprocessing, feature engineering, leakage, splits and pipelines
- Linear/logistic regression, trees, random forests, SVM, KNN, Naive Bayes, boosting, K-Means, DBSCAN and PCA
- Cross-validation, tuning, imbalance, explainability and error analysis
- Metrics: precision, recall, F1, ROC-AUC, MAE, RMSE and R-squared

Evidence of mastery

- One original project with EDA, baseline, pipeline, model comparison, tuning, explainability and error analysis
- MLflow experiment records and a reproducible training command
- A short model card covering data, metrics, limitations and intended use

Recommended use of your broad DS/ML/DL/NLP bootcamp: study Python libraries, math, classical ML and projects. Skip repeated basic NLP, RAG, LangChain and introductory agent sections.

Phase 6 - Deep Learning with PyTorch

Suggested duration	7-9 weeks
Outcome	Understand training and fine-tuning deeply enough to build and debug neural systems.

Learn in this order

- Tensors, autograd, Dataset/DataLoader and modules
- Forward pass, backpropagation, activations, losses and optimizers
- Training/validation loops, checkpoints and reproducibility
- Regularization, batch normalization, schedulers and mixed precision
- CNNs and transfer learning
- RNN/LSTM fundamentals as historical sequence models
- Attention and transformer architecture
- Efficient fine-tuning concepts, including LoRA/QLoRA theory beyond API usage

Evidence of mastery

- Train and evaluate one image or text model
- Serve the model through FastAPI
- Explain every part of a training loop and diagnose overfitting, underfitting and unstable loss

Phase 7 - Production Infrastructure, Linux and AWS

Suggested duration	6-8 weeks
Outcome	Deploy and operate an AI service rather than only demonstrate it locally.

Learn in this order

- Linux files, permissions, processes, ports, logs and shell workflows
- Git branching, pull requests, reviews and release tags
- Dockerfiles, Compose, networking, volumes, multi-stage builds and debugging
- GitHub Actions for linting, tests, security checks, image build and deployment
- AWS IAM, S3, Secrets Manager, ECR, ECS/App Runner, RDS, ElastiCache, CloudWatch, API Gateway/Lambda and Bedrock
- Load balancing, autoscaling, health checks, rollback and cost monitoring
- Kubernetes basics only after Docker and one AWS deployment

Evidence of mastery

- A Dockerized service deployed to AWS
- CI/CD that tests, builds and deploys
- Dashboards or alerts for errors, latency and service health
- A runbook covering deployment, rollback, incident diagnosis and secret rotation

Phase 8 - Production Agent Security, Evaluation and Reliability

Suggested duration	5-7 weeks
Outcome	Turn your flagship agent into a trustworthy production system.

Learn in this order

- Golden datasets, deterministic assertions, LLM-as-a-judge limitations and regression testing
- RAG metrics: retrieval quality, relevance, faithfulness and citation correctness
- Agent metrics: tool-call accuracy, trajectory quality, completion rate, latency and cost
- LangSmith or OpenTelemetry-style traces; logs, metrics and spans
- Input/output guardrails, prompt-injection defense and sensitive-data controls
- Provider gateways, routing, fallbacks, budgets, caching and rate limits
- PyRIT-style red teaming and threat modeling
- Retries, exponential backoff, circuit breakers, idempotency and dead-letter handling

Evidence of mastery

- Automated evaluation suite run in CI
- Red-team report and mitigations
- Fallback and recovery tests for LLM/tool/database failures
- A production dashboard for latency, errors, tokens, cost and quality

Phase 9 - AI System Design and Interview Sprint

Suggested duration	4-6 weeks
Outcome	Communicate architecture, trade-offs and production decisions confidently.

Learn in this order

- Design enterprise RAG, customer-support agent, deep research system, document platform and inference service
- Capacity estimation, latency budgets, cost models and reliability targets
- Stateful versus stateless services, queues, caching and event-driven systems
- Data privacy, tenant isolation, auditability and tool permissions
- Model selection, build-versus-buy decisions and fallback strategy
- Behavioural STAR stories: ownership, failure, disagreement, learning and delivery

Evidence of mastery

- Create 8 one-page system-design answers
- Complete mock rounds for Python/DSA, ML, LLM/RAG/agents, system design and behavioural interviews
- Prepare a 10-minute and a 2-minute explanation of your flagship project
- Tailor the resume to measurable engineering outcomes rather than course certificates

Flagship Portfolio Project

Build one serious system through the entire roadmap instead of creating many disconnected tutorial apps.

Recommended project: Enterprise Project Allocation and Knowledge Agent

Workflow	LangGraph state, conditional routing, tools, human approval, checkpointing and streaming
Knowledge	Document ingestion, hybrid retrieval, reranking, citations and access filters
Data	PostgreSQL/pgvector, Redis and S3-style storage
Service	FastAPI, async endpoints, authentication, authorization and feedback API
Quality	Golden dataset, retrieval/answer/trajectory evaluation and regression tests
Security	Guardrails, prompt-injection tests, tenant isolation, secrets and audit logs
Reliability	Timeouts, retries, idempotency, provider fallback, caching and rate limits
Operations	Docker, CI/CD, AWS deployment, monitoring, alerts and rollback runbook

Portfolio evidence to publish

- Architecture diagram and decision log
- Short demo video and deployed endpoint
- Automated test and evaluation reports
- Performance, cost and failure-test results
- Threat model and security controls
- README with setup, deployment, limitations and future improvements

Weekly Study Template

Core learning	10-12 hours	Focused lectures/docs only for the current phase
Hands-on building	16-20 hours	Implement, modify and integrate into the flagship project
DSA	5-6 hours	One daily problem session with explanation and tests
Revision/interview	3-4 hours	Recall notes, design questions and verbal explanations
Documentation	2 hours	README, architecture decisions and weekly progress log

The six-step learning loop

- Learn: understand the concept and its purpose.
- Rebuild: close the tutorial and reproduce it independently.
- Modify: change the model, data, state, tool or failure behavior.
- Integrate: add it to your primary project.
- Test: normal, invalid, timeout, injection, unavailable provider and high-load cases.
- Explain: document why you chose the design and its trade-offs.

Top-Company Readiness Checklist

Apply when most of these statements are true. You do not need perfection.

- I can solve common medium Python coding problems and explain complexity.
- I write modular, typed, async Python with tests and logging.
- I can design PostgreSQL schemas and write joins, CTEs and window functions.
- I can build authenticated, streaming FastAPI services.
- I understand classical ML workflows, metrics, leakage and error analysis.
- I can explain and implement a PyTorch training loop.
- I can build stateful LangGraph workflows without a tutorial.
- I can design advanced retrieval, reranking, citations and access control.
- I can evaluate RAG answers and agent trajectories using a versioned dataset.
- I can defend against prompt injection and protect sensitive data.
- I can Dockerize, deploy and monitor an application on AWS.
- I understand retries, idempotency, queues, caching and provider fallback.
- I can design an AI system and discuss scale, latency, cost, security and reliability.
- I can explain one production project deeply, including failures and trade-offs.

Final rule: After these courses, do not add another broad Agentic AI course. Fill specific gaps with documentation and focused resources, while continuously upgrading and operating your flagship project.

What not to prioritize yet

- Training foundation models from scratch
- Advanced CUDA kernel programming
- Expert-level Kubernetes administration
- Mastering every cloud, vector database or agent framework
- Research-level mathematics before you can ship and operate one system
- Collecting certificates without independent engineering evidence

Sources used for personalization

This roadmap was tailored from your supplied AI.txt and MLE.txt notes, including your completed Ed Donner content, current stopping point, planned LangGraph/security courses, and broader AI + ML engineering goals.